

گزارش پروژه سیستم مدیریت پرونده‌های پلیس (L.A Noire)

پروژه برنامه‌نویسی وب

۱. معرفی پروژه

این پروژه یک ****سیستم مدیریت پرونده‌های پلیس**** (LAPD Case Management System) است که در قالب پروژه درس برنامه‌نویسی وب طراحی و پیاده‌سازی شده است.

۱.۱ فناوری‌های مورد استفاده

بخش	فناوری
Backend	Django 6.0.2، Django REST Framework، JWT
Frontend	React 19، TypeScript، Vite، Shadcn UI
پایگاه داده	SQLite (توسعه)، PostgreSQL (آماده برای تولید)
مستندسازی API	Swagger/OpenAPI (drf-spectacular)

۱.۲ قابلیت‌های اصلی

- احراز هویت و مدیریت نقش‌ها (RBAC)
- مدیریت پرونده‌ها و شکایات
- مدیریت مدارک و شواهد
- تخته کارآگاه (Detective Board) با React Flow
- لیست تحت تعقیب‌ها (Most Wanted)
- مدیریت دادگاه‌ها (Trials)

۲. مسئولیت‌ها و کارهای انجام‌شده توسط هر عضو

< ****توصیه:**** این بخش را به صورت قدم‌به‌قدم همگام با پیشرفت پروژه در ویکی گیتلَب/گیتهاب به‌روزرسانی کنید.

۲.۱ (عضو اول: ۲۰) Setyhoseini (کامیت)

| ردیف | کار انجام شده |

|-----|

| ۱ | (Initial commit) ایجاد مخزن اولیه و ساختار پروژه |

| ۲ | submodule) افزودن فرانت‌اند به صورت فایل‌های عادی (نه |

| ۳ | JWT و Axios پیاده‌سازی ورود و ثبت‌نام با |

| ۴ | به‌روزرسانی سیستم احراز هویت و مسیرهای محافظت‌شده |

| ۵ | و آمار داشبورد API اتصال به RBAC، افزودن |

| ۶ | صفحات پرونده‌ها، شکایات و مدارک |

| ۷ | گزارش‌ها، تخته کارآگاه، ادمین و بررسی نقش Most Wanted، صفحات

مسیرها |

| ۸ | و افزودن ویژگی‌های باقی‌مانده UI تکمیل |

| ۹ | افزودن فایل‌های ادمین برای مدل‌ها |

| ۱۰ | رفع مشکلات جریان پرونده‌ها و نقش‌ها |

| ۱۱ | افزودن منطق تخصیص تصادفی کارآموز به شکایت |

| ۱۲ | react-flow پیاده‌سازی تخته کارآگاه با |

| ۱۳ | react-flow و تغییر رنگ خطوط اتصال accounts رفع باگ مدل |

۲.۲. عضو دوم: ۸) kimia (کامیت)

| ردیف | کار انجام شده |

|-----|

| ۱ | evidence، cases و accounts پیاده‌سازی مدل‌های |

| ۲ | trials و accounts رفع مشکلات ماژول |

| ۳ | Swagger/OpenAPI با drf-spectacular افزودن مستندات |

| ۴ | (role changes) رفع مشکلات نقش‌ها |

| ۵ | و منطق UI رفع مشکلات نقش‌ها و مجوزها در |

| ۶ | (merge) ادغام شاخه‌ها |

۳. قراردادهای توسعه

۳.۱. قرارداد نامگذاری (Naming Conventions)

(Backend (Python/Django

| نوع | قرارداد | مثال |

|-----|

```

| مدل‌ها | PascalCase | `Case`, `CrimeReport`, `Evidence`,
`User`, `Role` |
| ViewSet‌ها | PascalCase + ViewSet | `CaseViewSet`,
`EvidenceViewSet` |
| Serializer‌ها | PascalCase + Serializer | `CaseSerializer`,
`UserSerializer` |
| توابع و متغیرها | snake_case | `has_role()`,
`get_all_permissions()` |
| فایل‌ها | lowercase با underscore | `models.py`, `views.py`,
`serializers.py` |

```

Frontend (TypeScript/React)

```

| نوع | قرارداد | مثال |
|-----|-----|-----|
| کامپوننت‌ها | PascalCase | `CaseCreateDialog`,
`ComplaintApproveDialog` |
| توابع | camelCase | `getCases()`, `createCase()` |
| اینترفیس‌ها و تایپ‌ها | PascalCase | `Case`, `CrimeReport`,
`CreateCasePayload` |
| فایل‌های مسیر | kebab-case | `submit-complaint.tsx`,
`most-wanted/index.tsx` |
| ثابت‌ها | UPPER_SNAKE_CASE | `STATUS_COLORS`, `ROLE_CHOICES`
|

```

۳.۲. قالب پیام‌های کامیت (Commit Message Format)

بر اساس تاریخچه کامیت‌های پروژه، الگوهای زیر استفاده شده است:

```

| پیشوند | کاربرد | مثال |
|-----|-----|-----|
| `feat:` | ویژگی جدید | `feat: add RBAC permissions, wire API endpoints` |
| `fix` / `Fix:` | رفع باگ | `fix role permissions UI and logic` |
| `Added` | افزودن قابلیت | `Added admin files for models` |
| `Implemented` | پیاده‌سازی | `Implemented detective board using react-flow` |
| `Minor fix` / `Minor fixes` | اصلاحات جزئی | `Minor fixes in backend` |

```

****توصیه برای آینده: استفاده از Conventional [Commits] (https://www.conventionalcommits.org) برای یکنواختی بیشتر:**

````

<type>(<scope>): <description>

[optional body]

[optional footer]

````

****انواع پیشنهادی**** `feat`, `fix`, `docs`, `style`, `refactor`,
`test`, `chore`

۳.۳. استانداردهای کد

- ****Backend**** پیروی از PEP 8 و سبک Django
- ****Frontend**** TypeScript و ESLint، Prettier برای type-safety
- ****PR ارسال**** اجرای `pnpm lint` و `pnpm format` و `pnpm build`

۳.۴. ساختار شاخه‌ها (Branching)

- شاخه اصلی: `base` (یا `main`)
- برای ویژگی جدید: `feature/نام-ویژگی`
- برای رفع باگ: `fix/توضیح-باگ`

گزارش پروژه سیستم مدیریت پرونده‌های پلیس (L.A Noire)

پروژه برنامه‌نویسی وب

۱. معرفی پروژه

این پروژه یک ****سیستم مدیریت پرونده‌های پلیس**** (LAPD Case Management System) است که در قالب پروژه درس برنامه‌نویسی وب طراحی و پیاده‌سازی شده است.

۱.۱. فناوری‌های مورد استفاده

بخش	فناوری

Backend**	Django 6.0.2، Django REST Framework، JWT**
Frontend**	React 19، TypeScript، Vite، Shadcn UI**
پایگاه داده	SQLite (توسعه)، PostgreSQL (آماده برای تولید)
مستندسازی	Swagger/OpenAPI (drf-spectacular API**

۱.۲. قابلیت‌های اصلی

- احراز هویت و مدیریت نقش‌ها (RBAC)
- مدیریت پرونده‌ها و شکایات
- مدیریت مدارک و شواهد
- تخته کارآگاه (Detective Board) با React Flow
- لیست تحت تعقیب‌ها (Most Wanted)
- مدیریت دادگاه‌ها (Trials)

۲. مسئولیت‌ها و کارهای انجام‌شده توسط هر عضو

< **توصیه:** این بخش را به صورت قدم‌به‌قدم همگام با پیشرفت پروژه در ویکی گیت‌لب/گیت‌هاب به‌روزرسانی کنید.

۲.۱. عضو اول: (۲۰ Setyhoseini کامیت)

ردیف	کار انجام‌شده
۱	ایجاد مخزن اولیه و ساختار پروژه (Initial commit)
۲	افزودن فرانت‌اند به صورت فایل‌های عادی (submodule نه)
۳	پیاده‌سازی ورود و ثبت‌نام با Axios و JWT
۴	به‌روزرسانی سیستم احراز هویت و مسیرهای محافظت‌شده
۵	افزودن RBAC، اتصال به API و آمار داشبورد
۶	صفحات پرونده‌ها، شکایات و مدارک
۷	صفحات Most Wanted، گزارش‌ها، تخته کارآگاه، ادمین و بررسی نقش
مسیرها	
۸	تکمیل UI و افزودن ویژگی‌های باقی‌مانده
۹	افزودن فایل‌های ادمین برای مدل‌ها
۱۰	رفع مشکلات جریان پرونده‌ها و نقش‌ها
۱۱	افزودن منطق تخصیص تصادفی کارآموز به شکایت
۱۲	پیاده‌سازی تخته کارآگاه با react-flow
۱۳	رفع باگ مدل accounts و تغییر رنگ خطوط اتصال react-flow

۲.۲. عضو دوم: ۸) kimia (کامیت)

ردیف	کار انجام شده
۱	پیاده سازی مدل های accounts, cases و evidence
۲	رفع مشکلات مازول accounts و trials
۳	افزودن مستندات Swagger/OpenAPI با drf-spectacular
۴	رفع مشکلات نقش ها (role changes)
۵	رفع مشکلات نقش ها و مجوزها در UI و منطق
۶	ادغام شاخه ها (merge)

۳. قراردادهای توسعه

۳.۱. قرارداد نامگذاری (Naming Conventions)

(Backend (Python/Django

نوع	قرارداد	مثال
مدل ها	`Case`, `CrimeReport`, `Evidence`, `User`, `Role`	PascalCase
ViewSet ها	`CaseViewSet`, `EvidenceViewSet`	PascalCase + ViewSet
Serializer ها	`CaseSerializer`, `UserSerializer`	PascalCase + Serializer
توابع و متغیرها	`has_role()`, `get_all_permissions`	snake_case
فایل ها	lowercase با `models.py`, `views.py`, `serializers.py`	underscore

(Frontend (TypeScript/React

نوع	قرارداد	مثال
کامپوننت ها	`CaseCreateDialog`, `ComplaintApproveDialog`	PascalCase

```

| `()camelCase | `getCases()`, `createCase | توابع |
PascalCase | `Case`, `CrimeReport`, | اینترفیس‌ها و تایپ‌ها |
| ``CreateCasePayload
kebab-case | `submit-complaint.tsx`, | فایل‌های مسیر |
| ``most-wanted/index.tsx
`UPPER_SNAKE_CASE | `STATUS_COLORS`, `ROLE_CHOICES | ثابت‌ها |
|

```

۳.۲. قالب پیام‌های کامیت (Commit Message Format)

بر اساس تاریخچه کامیت‌های پروژه، الگوهای زیر استفاده شده است:

```

| پیشوند | کاربرد | مثال |
|-----|-----|-----|
feat: add RBAC permissions, wire API | ویژگی جدید | `:feat` |
| `endpoints
fix role permissions UI and | رفع باگ | `:fix` / `Fix` |
| `logic
| `Added admin files for models` | افزودن قابلیت | `Added` |
Implemented detective board | پیاده‌سازی | `Implemented` |
| `using react-flow
Minor fixes in | اصلاحات جزئی | `Minor fix` / `Minor fixes` |
| `backend

```

****توصیه برای آینده: استفاده از Conventional Commits****
 (/Commits] (<https://www.conventionalcommits.org>)
 برای یکنواختی بیشتر:

```

...
<type>(<scope>): <description>

[optional body]
[optional footer]
...

```

****انواع پیشنهادی: ****
 `feat`, `fix`, `docs`, `style`, `refactor`, `test`, `chore`

۳.۳. استانداردهای کد

- **:Backend** پیروی از PEP 8 و سبک Django
- **:Frontend** ESLint، Prettier و TypeScript برای type-safety
- **:PR ارسال** اجرای `pnpm lint` و `pnpm format`
- `build`

۳.۴ ساختار شاخه‌ها (Branching)

- شاخه اصلی: `base` (یا `main`)
- برای ویژگی جدید: `feature/نام-ویژگی`
- برای رفع باگ: `fix/توضیح-باگ`

۳.۵ Pull Request

- استفاده از قالب PR موجود در `github/PULL\_REQUEST\_TEMPLATE.md`
- ارجاع به issue مرتبط در صورت وجود
- بررسی CI قبل از merge

۴ نحوه مدیریت پروژه (چگونگی تولید و تقسیم وظایف)

۴.۱ ابزارها و بستر همکاری

ابزار	کاربرد
Git / GitHub	کنترل نسخه و میزبانی مخزن
base شاخه	شاخه اصلی توسعه
Pull Request	بررسی و ادغام تغییرات
Issue Templates	گزارش باگ و درخواست ویژگی

۴.۲ فرایند تولید و تقسیم وظایف

1. تقسیم بر اساس تخصص: بر اساس تاریخچه کامیت‌ها، وظایف بک‌اند (مدل‌ها، API، Swagger) و فرانت‌اند (UI، صفحات، تخته کارآگاه) بین اعضا تقسیم شده است.
2. جریان کار: توسعه روی شاخه‌های جداگانه، سپس merge به شاخه `base`.
3. همگام‌سازی: استفاده از merge برای یکپارچه‌سازی کد اعضا.

4. **بررسی کد: ** استفاده از قالب PR برای توضیح تغییرات و چکلیست قبل از merge.

۵. موجودیت‌های کلیدی سامانه

< هر موجودیت با دلیلی برای وجودش در سیستم توضیح داده شده است.

ردیف	موجودیت	ماژول	دلیل وجود
۱	accounts**	User**	کاربران سیستم (پلیس، ادمین، شاکی و ...): پایه احراز هویت و دسترسی
۲	accounts**	Role**	تعریف نقش‌ها و مجوزها برای کنترل دسترسی (RBAC)
۳	accounts**	Person**	اشخاص حقیقی درگیر در پرونده‌ها (شاکی، شاهد، مظنون) بدون وابستگی به User
۴	cases**	Case**	واحد اصلی کار؛ هر پرونده با شماره، وضعیت و اولویت
۵	cases**	CrimeReport**	شکایت/گزارش جرم با گردش کار تأیید (pending → approved/returned)
۶	cases**	Report**	گزارش‌های مختلف (شکایت، گشت، پزشکی قانونی) مرتبط با پرونده
۷	evidence**	Evidence**	مدارک و شواهد پرونده با زنجیره نگهداری و وضعیت
۸	accounts/cases**	Suspect**	مظنونین با وضعیت (تحت تعقیب، تعقیب فوری، دستگیرشده، آزاد)
۹	accounts**	CasePerson**	ارتباط Person با Case و نقش (شاکی، شاهد، مظنون، قربانی)
۱۰	board**	CaseBoard**	ذخیره وضعیت تخته کارآگاه (نودها و یال‌ها) برای هر پرونده
۱۱	trials**	Trial**	جلسات دادگاه، رأی و شهود مرتبط با پرونده
۱۲	cases**	Attachment**	فایل‌های پیوست به شکایات و گزارش‌ها

موجودیت‌های تخصصی Evidence

موجودیت	کاربرد
Testimony	شهادت متصل به Evidence
BiologicalEvidence	شواهد زیستی با بررسی پزشکی قانونی

| **VehicleEvidence** | شواهد مربوط به خودرو (پلاک، VIN) |
| **PhotoEvidence / VideoEvidence** | شواهد تصویری و ویدیویی |

۶. پکیج‌های NPM کلیدی (حداکثر ۶ مورد)

ردیف	پکیج	نسخه	کارکرد	توجیه استفاده
۱	tanstack/react-query**	^5.90@**	کش، واکشی و به‌روزرسانی داده از API	کاهش درخواست تکراری، مدیریت loading و error، invalidate و refetch خودکار
۲	tanstack/react-router**	^1.141@**	مسیریابی مبتنی بر فایل و type-safe	مسیرهای محافظت‌شده بر اساس نقش، lazy loading و کد تمیزتر
۳	axios**	^1.13**	ارسال درخواست HTTP به بک‌اند	interceptors برای JWT، مدیریت خطا و cancel کردن درخواست‌ها
۴	xyflow/react**	^12.3@**	گراف و نمودار تعاملی (نود و یال)	پیاده‌سازی تخته کارآگاه با نمایش روابط بین موجودیت‌ها
۵	zustand**	^5.0**	مدیریت state سراسری سبک	نگهداری وضعیت احراز هویت و تنظیمات بدون پیچیدگی Redux
۶	zod**	^4.2**	اعتبارسنجی schema و type inference	اعتبارسنجی فرم‌ها و داده‌های API با TypeScript

۷. چند نمونه کد تولید شده توسط هوش مصنوعی

```
# فایل: yourapp/migrations/0003_auto_....py
from django.db import migrations, models

def update_emails(apps, schema_editor):
    # 直接从 models  دریافت می‌کنیم، نه apps.get_model مدل را از طریق
    import
    Person = apps.get_model('yourapp', 'Person')
    for person in Person.objects.all():
        # فرض می‌کنیم می‌خواهیم ایمیل‌ها را به حروف کوچک تبدیل کنیم
        person.email = person.email.lower()
        person.save()

class Migration(migrations.Migration):
    dependencies = [
        ('yourapp', '0002_previous_migration'), # وابستگی به
    ]
    # مایگریشن قبلی
```

```

        operations = [
            migrations.RunPython(update_emails),
        ]
# accounts/views.py
from rest_framework import generics, permissions, status
from rest_framework.response import Response
from rest_framework.views import APIView
from rest_framework_simplejwt.tokens import RefreshToken
from django.contrib.auth import authenticate
from .serializers import UserSerializer, RegisterSerializer,
LoginSerializer
from .models import User

class RegisterView(generics.CreateAPIView):
    queryset = User.objects.all()
    permission_classes = (permissions.AllowAny,)
    serializer_class = RegisterSerializer

    def post(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        user = serializer.save()

        # ايجاد توكن JWT
        refresh = RefreshToken.for_user(user)

        return Response({
            'user': UserSerializer(user).data,
            'refresh': str(refresh),
            'access': str(refresh.access_token),
        }, status=status.HTTP_201_CREATED)

class LoginView(APIView):
    permission_classes = (permissions.AllowAny,)

    def post(self, request):
        serializer = LoginSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

```

```

identifier = serializer.validated_data['identifier']
password = serializer.validated_data['password']

# احراز هویت با بک‌اند سفارشی
user = authenticate(request, username=identifier,
password=password)

if user is not None:
    refresh = RefreshToken.for_user(user)
    return Response({
        'user': UserSerializer(user).data,
        'refresh': str(refresh),
        'access': str(refresh.access_token),
    })
else:
    return Response({
        'error': 'اطلاعات ورود صحیح نیست'
    }, status=status.HTTP_401_UNAUTHORIZED)

class ProfileView(generics.RetrieveUpdateAPIView):
    permission_classes = (permissions.IsAuthenticated,)
    serializer_class = UserSerializer

    def get_object(self):
        return self.request.user

```

۷.۱ ضعف ها و نقاط قوت هوش مصنوعی برای فرانت اند نقاط قوت

جنبه	توضیح
سرعت تولید کد	تولید سریع کامپوننت‌ها، فرم‌ها و صفحات بر اساس توضیحات متنی
کاهش تکرار	نوشتن کدهای تکراری (مثل CRUD، لیست‌ها، فرم‌ها) با سرعت بیشتر
یادگیری و مستندسازی	توضیح کد، API‌ها و ساختار پروژه‌های جدید
رفع باگ	پیشنهاد راحل برای خطاها و اصلاح کد بر اساس پیام خطا
تبدیل ایده به کد	تبدیل توضیحات به کد اولیه برای نمونه‌سازی سریع
پیشنهاد الگوها	پیشنهاد الگوهای رایج (مثل routing، state management، فرم‌ها)

نقاط ضعف

جنبه	توضیح
عدم آگاهی از context پروژه	ممکن است با معماری، قراردادهای و استایل کدنویسی پروژه شما هماهنگ نباشد
کدهای قدیمی یا نادرست	احتمال استفاده از APIها، نسخه‌ها یا الگوهای منسوخ یا اشتباه
کمبود خلاقیت در UX	طراحی‌های معمولی و کم‌تنوع، نه راحل‌های خلاقانه برای UX
مشکلات دسترسی (Accessibility)	ممکن است screen reader و ARIA، keyboard navigation را درست رعایت نکند
عملکرد و بهینه‌سازی	احتمال پیشنهاد کدهای غیر بهینه (re-render زیاد، bundle بزرگ و ...)
وابستگی به وابستگی‌ها	ممکن است پکیج‌های اضافی یا نامناسب پیشنهاد دهد
نقص در منطق پیچیده	در منطق پیچیده کسب‌وکار و edge caseها احتمال اشتباه بیشتر است
امنیت	ممکن است آسیب‌پذیری‌های امنیتی (مثل XSS) را در نظر نگیرد

۷.۲. ضعف‌ها و نقاط قوت هوش مصنوعی برای توسعه بک اند نقاط قوت

جنبه	توضیح
تولید سریع API	ساخت سریع endpointها، serializerها و viewها بر اساس مدل‌ها
طراحی مدل و دیتابیس	پیشنهاد مدل‌ها، روابط و migrationها
منطق کسب‌وکار ساده	پیاده‌سازی CRUD، فیلترها و جستجوهای ساده
مستندسازی	تولید Swagger/OpenAPI، docstring و توضیح endpointها
رفع خطا	تحلیل traceback و پیشنهاد اصلاح کد
امنیت پایه	پیشنهاد authentication، authorization و validation ساده
الگوهای رایج	استفاده از الگوهای Repository، Service Layer و مشابه آن
تست	نوشتن unit test و integration test برای endpointها

نقاط ضعف

جنبه	توضیح
منطق پیچیده	در business logic پیچیده و edge caseها احتمال اشتباه بیشتر است
عملکرد و مقیاس‌پذیری	ممکن است N+1 query، index، نادرست یا طراحی ضعیف برای scale پیشنهاد دهد
امنیت پیشرفته	احتمال نادیده گرفتن rate limiting، CSRF، SQL injection و آسیب‌پذیری‌های خاص

تراکنش‌ها	مدیریت درست transaction، rollback و race condition نیاز به بررسی دستی دارد
وابستگی به فریمورک	ممکن است API‌ها یا الگوهای قدیمی یا نامتناسب با نسخه فریمورک پیشنهاد دهد
معماری	طراحی معماری کلی سیستم و microservice‌ها نیاز به تجربه انسانی دارد
داده و یکپارچگی	طراحی migration‌های پیچیده و حفظ یکپارچگی داده‌ها حساس است
مستندسازی واقعی	مستندات ممکن است با رفتار واقعی کد هماهنگ نباشد

۸. منابع و مستندات تکمیلی

- `backend/BACKEND\_IMPLEMENTATION.md` - مستندات تفصیلی بک‌اند
 - `backend/README.md` - راهنمای بک‌اند
 - `frontend/README.md` - راهنمای فرانت‌اند
 - `frontend/.github/CONTRIBUTING.md` - راهنمای مشارکت
 - (Swagger) `/api/docs/` (API Docs) و (ReDoc) `/api/redoc/`
 -

۹. نیازسنجی‌های ابتدایی و نقاط ضعف آن‌ها:
 - ما در ابتدای راه‌اندازی کلی پروژه تصمیم گرفتیم که برای بخش‌های بک‌اند و فرانت‌اند دو ریویو جدا داشته باشیم. اما این موضوع روند کار گروهی و هماهنگی و بالا آوردن پروژه را کند می‌کرد، و همچنین برای داکرایز کردن پروژه، به نظرم ساختار تک‌ریویو روند کار را سهولت می‌داد. بنابراین تصمیم گرفتیم ریویوی جدید شامل فولدرهایی جدا برای فرانت‌اند و بک‌اند بسازیم، و تغییرات را آن جا اعمال کنیم.
 - برای معماری مدل‌ها، در ابتدا تصمیم گرفتیم که موجودیت شواهد/مدارک را تحت موجودیت پرونده طراحی کنیم. اما متوجه شدیم این طراحی روند توسعه را سخت‌تر می‌کند و بنابراین موجودیت شواهد را جدا طراحی کردیم.
 - در ابتدا برای طراحی نقش‌ها روابط کاربرها و نقش‌ها را به صورت M2M در نظر گرفته بودیم؛ اما بعداً برای طراحی داشبورد مازولار برای هر نقش به مشکل خوردیم. در نتیجه تولید و ویرایش نقش‌ها را طوری پیاده‌سازی کردیم که در پنل ادمین به صورت یک به یک به کاربرها داده شود.